CollabHub - Project Report

1. Project Overview

CollabHub is a real-time team collaboration and communication web application that I developed to make team messaging, file sharing, and video calling easy, organized, and accessible from any browser. In this system, team members can join or create teams, send instant messages in team channels or private 1-on-1 direct chats, upload files/media securely, react to messages with emojis, and start live WebRTC video/audio calls without installing any extra software. There are two primary types of users in the system: the Member who collaborates and chats, and the Admin who manages users, moderates team channels, and oversees system performance. I built this project using Node.js, Express, and Supabase PostgreSQL with Sequelize ORM for the backend, Socket.io and WebRTC for real-time signaling, with simple HTML, CSS, and JavaScript on the frontend, all deployed on Render.

2. Problem Statement

In most modern remote and hybrid teams, communication is fragmented across multiple apps — using separate tools for team messaging, direct chats, file sharing, and video calling. Switching back and forth between different platforms causes distraction, lost context, and increased friction in daily collaboration. Additionally, team managers often lack a single unified dashboard to monitor active channels, handle content moderation, or manage user access permissions. I built CollabHub to solve exactly these problems to give team members an all-in-one, lightweight collaboration workspace and give admins structured control over users and team channels.

3. My Approach & How I Solved It

To solve this, I built a structured micro-serviced architecture where real-time socket events and REST API endpoints work seamlessly together. I used Node.js and Express for the backend API, Supabase PostgreSQL with Sequelize ORM to store relational data, and JWT for login and role-based access control so each user only sees the teams and channels they are authorized to access. For real-time messaging, typing status, and instant notification alerts, I integrated Socket.io. For audio and video calling, I built a WebRTC peer-to-peer signaling mechanism over WebSockets. For file uploads and avatar media, I used Multer with Cloudinary so files are stored securely in the cloud and accessible at any time.

4. Core Features

- Users can register, log in, manage their profile, and upload profile pictures directly stored on Cloudinary.
- Team members can create new teams or join existing ones using unique team invite codes.
- Structured channel messaging lets users send real-time text messages, attach images or documents, and add emoji reactions to messages.
- Private 1-on-1 direct messaging allows end-to-end communication with live read receipts and online presence indicators.
- In-browser WebRTC video and audio calls with mute, camera toggle, and real-time caller notification popups.
- Instant notifications alert users automatically whenever they receive direct messages, team mentions, or call invitations.
- Admins can manage all users (block/unblock, assign admin roles), moderate team channels, and view system metrics.
- Rate limiting and JWT authentication are applied to all API endpoints to prevent abuse and protect user data.
- Persistent message history stored in Supabase PostgreSQL enables quick search and retrieval.
- All routes are protected using role-based middleware.

5. Technologies & Tools Used

Area	Technology / Tool	What it does
Runtime	Node.js + Express	Runs the backend server and handles all API requests
Database	Supabase PostgreSQL	Stores all application data like users, teams, messages, and calls
ORM	Sequelize	Lets me write database queries using JavaScript instead of raw SQL
Real-Time Engine	Socket.io	Handles real-time messaging, typing indicators, and instant notifications
Video Calling	WebSockets + WebRTC	Enables real-time peer-to-peer audio/video streaming and signaling
Authentication	JWT + bcryptjs	Handles login and keeps passwords secure by hashing them
File Upload	Multer + Cloudinary	Accepts uploaded media/files and stores them safely in the cloud
Environment Config	dotenv	Keeps sensitive values like passwords and API keys out of the code
Dev Server	nodemon	Automatically restarts the server whenever I save a file
Frontend	Plain HTML + CSS + JS	Simple static pages with no heavy framework or build step needed
Client Storage	localStorage + sessionStorage	Saves the login token and user session info in the browser
Deployment	Render	Hosts the backend service and serves the frontend

6. APIs & Their Purpose

API / Service	Purpose
REST API (Express)	The main backend API that handles all requests from the frontend — login, team management, chat history, user profiles, and admin moderation. All routes are prefixed with /api/v1.
Socket.io Signaling	Manages real-time bi-directional events for channel chat (send-message), direct messages, typing indicators, and WebRTC call signaling (call-user, make-answer, ice-candidate).
JWT	Used to generate a login token when a user signs in. This token is sent with every request and socket connection so the backend knows who is making the request and what role they have.
bcryptjs	Used to hash passwords before saving them to the database, so even if the database is leaked, actual passwords are never exposed.
Cloudinary	Cloud storage service used to save uploaded avatars and chat attachments. When a user uploads a file, Multer sends it directly to Cloudinary and stores the file URL in PostgreSQL.
Multer	A middleware that handles multipart file uploads from the frontend form before passing them to Cloudinary.
Supabase PostgreSQL (via Sequelize)	The main cloud database where everything is stored — users, teams, team members, messages, direct messages, call logs, and notifications. Sequelize manages all queries and relationships.

7. System Architecture & Diagrams

A. How the API Works (Request-Response Flow)

```
Frontend (UI) -> Express Router (routes/v1) -> Auth Middleware -> Controller -> Service Layer -> Supabase PostgreSQL & Cloudinary
Frontend (UI) -> WebSockets -> Socket.io Server -> Chat Handler / WebRTC Signaling -> Broadcast / Peer Connection
```

B. Data Flow Diagram

```
User Payload -> Auth Verification (JWT) -> Controller -> Service Layer -> Supabase Database
File Upload -> Multer Middleware -> Cloudinary API -> Return CDN URL
Socket Event -> Socket.io Server -> Broadcast to Active Room Members
```

C. How User Workflows Work

```
User (Registers & Logs in) -> Join or Create Team -> Select Channel / Direct Chat
• Real-Time Group Chat (Messages, Reactions, Files)
• WebRTC Video / Audio Call (Caller sends signal -> Peer accepts)
• Saved in Supabase DB & Logs
```

D. Document & File Upload Architecture

```
Frontend Form Data -> Multer Middleware -> Upload Stream -> Cloudinary Cloud -> Return Secure URL ->
Controller -> Save URL in PostgreSQL 'messages' Table
```

8. What Happens When Someone Uses the App

• A user registers, logs in, fills out their profile details, and uploads a profile picture.

• The user creates a new team or joins an existing team using a shared team code.

• The user opens a team channel and starts chatting. Socket.io delivers messages instantly to all online team members while Sequelize saves the chat history in Supabase PostgreSQL.

• If the user wants to talk privately, they open Direct Messaging and start a 1-on-1 chat or initiate an in-browser WebRTC video/audio call.

• Every time a message is sent, file is shared, or call is initiated, the system automatically sends real-time notifications to the recipient.

• Admins can access the Admin Dashboard to view active users, manage team memberships, block inappropriate users, and monitor overall platform activity.

9. What Data the App Stores

User Table

Field	Type	Notes
id	UUID	Primary Key
name / email	String	User identity and unique login email
password	String	Encrypted password hashed with bcryptjs
role	String	User role (SUPER_ADMIN, MEMBER, admin, user)
profilePic	String	Profile picture Cloudinary URL
isBlocked	Boolean	Account suspension status

Team Table

Field	Type	Notes
id	UUID	Primary Key
name	String	Team name
description	String	Team description
ownerId	UUID	User ID of the team creator

TeamMember Table

Field	Type	Notes
id	UUID	Primary Key
userId / teamId	UUID	Links member user to a specific team
role	String	Role within the team (TEAM_ADMIN, MEMBER, admin, member)
lastReadAt	Date	Timestamp of last read message

Message Table

Field	Type	Notes
id	UUID	Primary Key
senderId / teamId	UUID	Links message to sender and team
content	Text	Text content of the message
fileUrl / fileType	String	Cloudinary attachment link and media type
isDeleted	Boolean	Soft delete flag

DirectMessage Table

Field	Type	Notes
id	UUID	Primary Key
senderId / recipientId	UUID	Links direct message to sender and recipient
content	Text	Text content of the direct message
isRead	Boolean	Read status indicator

Notification Table

Field	Type	Notes
id	UUID	Primary Key
userId	UUID	Recipient user ID
title / message	String	Headline and notification content
isRead	Boolean	Read status flag

10. Main Pages & Actions (Routes)

Area	Sample Routes	Access Level
Auth	POST /api/v1/auth/register POST /api/v1/auth/login	Public (Register / Login)
User Profile	GET /api/v1/user/profile PUT /api/v1/user/profile	Logged-in Users
Team	POST /api/v1/team GET /api/v1/team POST /api/v1/team/join	Logged-in Users
Team Chat	GET /api/v1/chat/messages/:teamId POST /api/v1/chat/message	Team Members
Direct Chat	GET /api/v1/direct/messages/:userId POST /api/v1/direct/message	Logged-in Users
Admin	GET /api/v1/admin/users PATCH /api/v1/admin/user/:id/block	Admin role only
Notifications	GET /api/v1/notification PATCH /api/v1/notification/:id/read	Logged-in Users

11. Deployment

I deployed the application on Render. The backend runs as a Node.js web service connected to a cloud-hosted Supabase PostgreSQL database using SSL encryption. The frontend is built using static HTML, CSS, and JavaScript and is deployed on Render as a Static Site at https://collabhub-1-whx9.onrender.com. Environment variables like PORT, JWT_SECRET, DATABASE_URL (Supabase Postgres Connection String), and Cloudinary keys (CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET) are set directly in the Render dashboard. On the client side, frontend/js/config.js is configured with the live Render backend URL (https://collabhub-qvx3.onrender.com) so all API and WebSocket requests connect smoothly.

12. Links

- **GitHub Repository**: https://github.com/Kuwarjibetha/CollabHub
- **Backend Live Service Link**: https://collabhub-qvx3.onrender.com
- **Frontend Live Website Link**: https://collabhub-1-whx9.onrender.com
- **User / Admin Email 1**: bethajikuwa@gmail.com / Password: Kuwarji@9934
- **User / Admin Email 2**: bethakuwarji@gmail.com / Password: Kuwarji@9934

13. Conclusion

CollabHub is a complete team collaboration and WebRTC communication system that unifies team messaging, direct chat, media sharing, and video calling into a single digital platform. By dividing responsibilities across Members and Admins, the platform ensures secure, real-time collaboration with complete accountability. Building this project helped me understand how to structure domain-driven Node.js applications, manage PostgreSQL relational databases using Sequelize, handle cloud uploads with Cloudinary, implement real-time WebSockets with Socket.io and WebRTC, and enforce strict role-based access control.